

Angular Forms

- Forms let users enter and send data.
- Angular helps connect form fields with app logic.

What is a Form?

A form is a group of fields where the user types information.

- Text fields like name and email
- Password fields for login
- Options with select, radio buttons, autocomplete
- A submit button to send the data

A form is how the app gets user input.

Why Use Forms in Angular?

Angular forms make input easier to manage.

- Automatically keep values in the component
- Show validation errors clearly
- Know when a field is touched or changed
- Prevent invalid data from being submitted

How Angular Forms Work

1. Define form fields in HTML.
2. Connect fields to Angular form logic.
3. Add validation rules.
4. Show messages when input is wrong.
5. Submit only when the form is valid.

Two Types of Angular Forms

Template-driven forms:

- Simple, good for beginners
- Use `FormsModule`
- Bind fields with `[(ngModel)]`

Reactive forms:

- More powerful, best for complex forms
- Use `ReactiveFormsModule`
- Define form structure in TypeScript

Template-Driven Form Example

This example is a login page.

- Username field
- Password field
- Required checks
- Password length check
- Show/hide password eye icon

It is easy to read and good for teaching.

Template-Driven Form Code Explained

```
<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">  
  <input name="username" required [(ngModel)]="model.username" #username="ngModel" />  
</form>
```

- `#loginForm="ngForm"` : this gives us the form state.
- `[(ngModel)]="model.username"` : this keeps input and code in sync.
- `#username="ngModel"` : this lets us check if username is valid.

Why We Choose Template-Driven Here

Use template-driven when:

- the form is small
- you want less TypeScript code
- the form is easy to write in the template

Template-Driven Form Use Cases

- Login forms
- Simple contact forms
- Small signup forms
- Quick prototypes

These forms usually do not need complex logic.

Reactive Form Example

This example is a student registration form.

- Name, email, age
- Gender radio buttons
- Course dropdown
- City autocomplete
- Hobby list using `FormArray`

It shows how to handle more complex data.

Reactive Form Code Explained

```
this.registrationForm = this.fb.group({  
  name: ['', [Validators.required, Validators.minLength(3)]],  
  email: ['', [Validators.required, Validators.email]],  
  gender: ['', Validators.required],  
  course: ['', Validators.required],  
  city: ['', Validators.required],  
  hobbies: this.fb.array([this.fb.control('', Validators.required)]),  
});
```

- `FormGroup` holds all fields together.
- `FormControl` is one field.
- `FormArray` stores many hobby inputs.

Reactive Template Code

```
<form [formGroup]="registrationForm" (ngSubmit)="onSubmit()">  
  <mat-select formControlName="course"></mat-select>  
  <input matInput formControlName="city" [matAutocomplete]="auto" />  
</form>
```

- `formControlName="course"` links the field to the form.
- `mat-autocomplete` shows city suggestions while typing.

FormArray for Hobbies

```
get hobbies(): FormArray {  
    return this.registrationForm.controls.hobbies as FormArray;  
}  
  
addHobby() {  
    this.hobbies.push(this.fb.control('', Validators.required));  
}
```

```
<div *ngFor="let hobbyControl of hobbyControls; let i = index">  
    <input matInput [formControl]="hobbyControl" />  
</div>
```

- Use `FormArray` when the user can add many items.
- Each hobby becomes a separate field in the form.

Why We Use Reactive Forms Here

Choose reactive when:

- the form is larger or more complex
- you need custom validations
- the fields can change dynamically
- you want full control in TypeScript

Reactive Form Use Cases

- Registration forms
- Survey forms with many questions
- Dynamic forms where fields are added or removed
- Forms with complex validation logic

Angular Material Input Controls

The example uses Material controls to improve the look.

- `mat-form-field` : wraps input fields.
- `mat-input` : styled text input.
- `mat-select` : dropdown menu.
- `mat-autocomplete` : suggestions while typing.
- `mat-radio-button` : choose one option.
- `mat-icon` : eye icon for password.

What is Form State?

Angular tracks the state of each field.

- `touched` : user entered then left the field.
- `untouched` : user has not focused the field.
- `pristine` : field value is unchanged.
- `dirty` : field value has changed.
- `valid` : field passes all rules.
- `invalid` : field fails a rule.

Using State in Template Forms

Use state to show messages only after the user interacts.

```
<mat-error *ngIf="password.errors?.['required'] && password.touched">  
  Password is required  
</mat-error>
```

This avoids showing errors before the user types.

Using State in Reactive Forms

Reactive forms use the form object to check field state.

```
<mat-error *ngIf="registrationForm.controls.email.invalid && registrationForm.controls.email.touched">  
  Enter a valid email.  
</mat-error>
```

Use `markAllAsTouched()` when the user clicks submit.

Easy Summary

- A form collects user data.
- Template-driven forms are easy and use `ngModel` .
- Reactive forms are more powerful and use `FormGroup` .
- Material controls make forms look better.
- Form state helps show errors at the right time.

What We Built in This App

Template-driven login form:

- simple form structure
- beginner-friendly bindings
- password visibility toggle

Reactive student form:

- more fields and validation
- select, autocomplete, radio buttons
- dynamic hobby fields with `FormArray`